

Esc. Tecnica Rocha — ANEP, Consejo de Educacion Tecnico Profesional (UTU)
3.º BT Tecnologias de la Informacion
Programacion Full-Stack

UTalent

Buscador de oportunidades laborales en Tecnologias de la Informacion para estudiantes y egresados de UTU

Cuarto Practico — 2026
Estudiante: Lucas Techera (C.I. 4.999.390-1)
Rocha, setiembre de 2026

Indice

1. Presentacion de UTalent.....	4
1.1 Que es UTalent.....	4
1.2 Problema que resuelve.....	4
1.3 Publico objetivo.....	4
1.4 Alcance.....	4
2. Funcionalidades principales.....	5
3. Fuentes de empleo utilizadas.....	6
3.1 Uruguay Concurso (uruguayconcurso.gub.uy) — publico.....	6
3.2 BuscoJobs Uruguay (buscojobs.com.uy) — privado.....	6
3.3 Por que endpoints JSON internos y no scraping de HTML.....	6
3.4 Fuentes descartadas o no incluidas.....	6
4. Stack tecnologico.....	7
4.1 Backend.....	7
4.2 Frontend.....	7
4.3 Por que Laravel y no PHP/SQL vanilla.....	7
5. Arquitectura: MVC + Strategy + Repository + Service.....	8
5.1 MVC (Laravel).....	8
5.2 Strategy — FuenteEmpleoInterface.....	8
5.3 Repository — acceso a persistencia.....	8
5.4 Service — BuscadorService.....	8
5.5 Un cuarto colaborador con su propia interfaz: NormalizadorIA.....	8
6. Diagrama de clases (actualizado contra el codigo real).....	9
7. Flujo general del sistema.....	10
7.1 Camino de actualizacion (en segundo plano).....	10
7.2 Camino de busqueda del usuario (siempre local).....	10
8. Flujo de actualizacion de ofertas.....	11
8.1 actualizarDesdeTermino(termino) — un termino puntual.....	11
8.2 actualizarFuentes() — barrido completo.....	11
8.3 Por que guardar es siempre updateOrCreate.....	11
9. Queue y Scheduler.....	12
10. API.....	13
10.1 Busqueda (solo lectura de datos locales).....	13
10.2 ABM (fuentes y sinonimos).....	13
11. Buscador y filtros.....	14
11.1 Por que no existe un filtro literal de “postulacion online”.....	14
12. Sinonimos.....	15
13. NormalizadorIA.....	16
14. Frontend.....	17
14.1 Paginas.....	17
14.2 Organizacion del JavaScript.....	17
14.3 Estado por URL.....	17
15. Accesibilidad y diseño responsive.....	18
16. Manejo de errores y resiliencia.....	19
17. Cierre de ofertas y por que requiere un barrido completo.....	20
17.1 Antigüedad de una oferta: señal visual, no cierre.....	20
18. Segunda fuente: demostracion real de Strategy.....	21
19. Tests y validaciones.....	22
19.1 Que no tiene test automatizado, y por que.....	22
20. Uso de Inteligencia Artificial (Ejercicio 3).....	23

A) IA como parte del producto: NormalizadorIA + OpenRouter.....	23
B) IA como herramienta de desarrollo: Claude Code.....	23
21. Instalacion y ejecucion.....	25
21.1 Instalacion inicial.....	25
21.2 Generar datos reales antes de una demo.....	25
21.3 Levantar el proyecto para desarrollo o demo.....	25
21.4 Ejecutar la suite de tests.....	25
22. Variables de entorno.....	26
23. Limitaciones conocidas.....	27
24. Conclusiones.....	28

1. Presentacion de UTalent

1.1 Que es UTalent

UTalent es un buscador de oportunidades laborales en Tecnologias de la Informacion que agrega en un unico lugar ofertas publicadas en distintos sitios de Uruguay. El usuario busca por titulo de puesto (por ejemplo “Programador” o “Soporte Tecnico”), aplica filtros y ve resultados combinados de mas de una fuente sin tener que visitarlas por separado.

1.2 Problema que resuelve

Un estudiante o egresado de UTU que busca su primer empleo en informatica hoy tiene que revisar por separado el portal estatal de concursos publicos y varios portales privados de empleo, cada uno con su propia forma de buscar y filtrar, para armarse una idea completa de que oportunidades existen. UTalent resuelve ese problema concreto: agrega ambos tipos de fuente, normaliza los datos a un formato comun y permite buscar y filtrar una sola vez.

1.3 Publico objetivo

Estudiantes de 3.º y 4.º BT de Tecnologias de la Informacion y egresados recientes de UTU que buscan su primera experiencia laboral en el area: perfiles junior, sin necesariamente experiencia previa formal, para quienes filtrar por sector publico/privado, salario visible o modalidad importa mas que para un perfil senior que ya tiene contactos en la industria.

1.4 Alcance

Segun la consigna de la Opcion 3, el alcance excluye explicitamente el registro de usuarios y la postulacion a una oferta desde el propio sistema: UTalent muestra y filtra ofertas, y deriva al usuario al sitio original para postularse (“Ver publicacion original”). Dentro de ese alcance, el proyecto fue mas alla del MVP minimo: en vez de una sola fuente se implementaron dos, con un mecanismo real de actualizacion periodica en segundo plano, cierre de ofertas que ya no aparecen en la fuente original, enriquecimiento opcional con IA y un panel de administracion para el propio motor de busqueda (fuentes y sinonimos).

2. Funcionalidades principales

- Búsqueda de ofertas por título de puesto, con expansión automática por sinónimos (ej.: buscar “programador” también encuentra ofertas tituladas “developer”).
- Filtros combinables: sector (público/privado), salario visible o no, modalidad laboral (Remoto/Híbrido/Presencial, cuando la fuente la informa) y departamento.
- Paginación de resultados y estado de la búsqueda (cargando, sin resultados, error) manejado en el frontend sin recargar la página.
- Detalle de una oferta puntual, incluyendo ofertas ya cerradas (se muestran igual, marcadas como no disponibles, en vez de desaparecer u ocultarse).
- Señal visual de “Oferta antigua” para publicaciones de más de 180 días, sin que eso afecte si la oferta sigue activa o no (ver sección 17).
- Actualización periódica automática de ofertas en segundo plano (cada 6 horas) y cierre de las que ya no aparecen en la fuente original.
- Enriquecimiento opcional de cada oferta con seniority y tecnologías detectadas por IA a partir del texto original.
- Panel de administración (sin autenticación, solo por URL directa) para dar de alta/baja fuentes de empleo y administrar los sinónimos de búsqueda.
- API REST propia (JSON) que expone toda la búsqueda y el ABM, consumida por el frontend vía Axios.

3. Fuentes de empleo utilizadas

3.1 Uruguay Concurso (uruguayconcurso.gub.uy) — publico

Portal oficial de concursos y llamados del Estado uruguayo. El sitio no publica una API documentada, pero su propio buscador interno consume un endpoint JSON (api-backend/llamados/find) que se identifico inspeccionando el trafico de red del sitio real, no adivinando. UTalent lo consulta con los mismos parametros que usa el sitio (termino, estados vigentes, rango de años) y traduce cada resultado a un formato comun (OfertaDTO).

3.2 BuscoJobs Uruguay (buscojobs.com.uy) — privado

Portal privado de empleo. Es una aplicacion Next.js cuyos resultados de busqueda no vienen en el HTML: los trae un endpoint JSON interno identificado por un “buildId” que cambia con cada despliegue del sitio. Por eso cada busqueda implica dos peticiones: una a la pagina de busqueda (para leer el buildId vigente del momento) y otra al endpoint de datos con ese id. Adicionalmente, cuando el listado devuelve una descripcion truncada (fenomeno verificado: el listado siempre corta a 150 caracteres), se hace una tercera peticion opcional al endpoint de detalle de esa oferta puntual para obtener el texto completo (ver seccion 18).

3.3 Por que endpoints JSON internos y no scraping de HTML

Ambas fuentes se investigaron antes de programar nada: se inspecciono el trafico de red real de cada sitio (pestaña Network del navegador) para encontrar el endpoint que el propio sitio usa internamente, y se confirmo con peticiones directas (curl) antes de escribir la Strategy correspondiente. Consultar ese endpoint en vez de parsear el HTML final da datos ya estructurados, es mas estable ante cambios visuales del sitio, y es mas rapido.

3.4 Fuentes descartadas o no incluidas

El diseño original (documentado en la primera version de este documento) contemplaba una segunda fuente privada ademas de BuscoJobs. Por alcance y tiempo se implemento una sola fuente privada: alcanza para demostrar que el patron Strategy funciona con mas de una fuente real (ver seccion 18), que es el objetivo pedagogico del ejercicio, sin que una tercera fuente aporte algo estructuralmente distinto.

4. Stack tecnologico

4.1 Backend

- PHP 8.2 + Laravel 12.
- Base de datos SQLite (un unico archivo, sin necesidad de un servidor de base de datos aparte).
- Eloquent ORM, Queue (driver database), Scheduler y el cliente HTTP (Http::) que Laravel trae de fabrica.
- Sin librerias externas de scraping ni de terceros para consumir las fuentes: solo el cliente HTTP de Laravel y expresiones regulares puntuales.

4.2 Frontend

- Blade (plantillas de Laravel) unicamente para la estructura de cada pagina.
- Tailwind CSS 4 para estilos, con tokens de color propios definidos via @theme (sin configuracion JS).
- JavaScript vanilla en modulos ES (sin React, Vue, Alpine, Livewire ni Inertia).
- Axios como unico cliente HTTP del frontend hacia la API.
- Vite como bundler, con un punto de entrada JS por pagina (bundles pequeños y enfocados).

4.3 Por que Laravel y no PHP/SQL vanilla

Se descarto PHP/SQL vanilla en favor de Laravel por los siguientes motivos:

- El foco del practico es la extraccion y consulta de datos, no la infraestructura. El proyecto necesita colas de trabajo (para actualizar ofertas en segundo plano sin bloquear al usuario), un scheduler (para relanzar la actualizacion periodicamente), un cliente HTTP para invocar un modelo de lenguaje, y un ORM para modelar relaciones entre fuentes, ofertas y sinonimos de busqueda. Laravel provee todo esto de fabrica (Queues, Scheduler, Http::, Eloquent), dejando el tiempo disponible para la logica de extraccion y normalizacion, que es el objetivo pedagogico real.
- Consistencia de convenciones: al tener varias fuentes de datos heterogeneas, conviene una estructura de carpetas y convenciones ya establecidas en vez de definirlas desde cero.
- Valor de aprendizaje adicional: suma un framework ampliamente usado en la industria, en linea con el aporte al proyecto de egreso que pide el ejercicio 3.

5. Arquitectura: MVC + Strategy + Repository + Service

El proyecto usa el MVC que Laravel impone por convencion (rutas → controladores → modelos Eloquent → vistas Blade) como esqueleto general, pero el MVC puro no resuelve bien un problema central de este proyecto: obtener datos de fuentes externas heterogeneas y cambiantes sin acoplar esa logica al resto de la aplicacion. Por eso se suman tres piezas adicionales.

5.1 MVC (Laravel)

Rutas (routes/web.php, routes/api.php) enrutan hacia controladores delgados (OfertaController, FuenteController, SinonimoController), que delegan en el Service o en los Repository segun corresponda y devuelven JSON (API) o una vista Blade (web). Las vistas Blade solo aportan estructura: ningun controlador inyecta datos de ofertas, fuentes o sinonimos hacia una vista; todo ese dato lo carga el propio JavaScript de cada pagina contra la API (ver seccion 14).

5.2 Strategy — FuenteEmpleoInterface

Cada fuente de empleo (Uruguay Concurso, BuscoJobs) implementa la misma interfaz FuenteEmpleoInterface. Agregar, quitar o reparar una fuente no requiere tocar el resto del sistema (principio abierto/cerrado): si un sitio cambia su formato, solo se ajusta su clase concreta. BuscadorService nunca sabe si esta hablando con Uruguay Concurso o con BuscoJobs, solo ve la interfaz.

5.3 Repository — acceso a persistencia

El acceso a la base de datos (ofertas, fuentes, sinonimos) se aísla detras de interfaces (OfertaRepositoryInterface, FuenteRepositoryInterface, SinonimoRepositoryInterface), separando de donde vienen los datos externos de como se guardan y consultan localmente. Esto tambien permite que la busqueda del usuario final nunca dependa de consultar una fuente externa en vivo: los resultados se persisten y refrescan por Jobs en segundo plano, haciendo el sistema tolerante a que una fuente falle o cambie de estructura en el momento exacto en que alguien esta buscando.

5.4 Service — BuscadorService

BuscadorService orquesta las Strategy, delega el enriquecimiento de texto libre a NormalizadorIAInterface y persiste via los Repository. Es la unica clase que conoce el flujo completo (consultar fuentes → guardar → normalizar → , en el barrido completo, cerrar lo que ya no aparece); no conoce ningun detalle propio de un sitio externo ni del proveedor de IA.

5.5 Un cuarto colaborador con su propia interfaz: NormalizadorIA

Aunque el titulo de esta seccion nombra tres patrones, en la practica hay un cuarto colaborador con el mismo espiritu que Strategy: NormalizadorIAInterface, con una unica implementacion real (OpenRouterNormalizadorIA). Se documenta aparte (seccion 13) porque su responsabilidad es distinta (enriquecer datos, no obtenerlos de una fuente), pero comparte la misma logica de aislamiento: BuscadorService nunca sabe que proveedor de IA hay detras.

6. Diagrama de clases (actualizado contra el código real)

A diferencia de la primera version de este documento —hecha antes de programar, como pide el ejercicio 1—, el diagrama que sigue se rehizo leyendo las clases reales del repositorio tal como quedaron al finalizar el proyecto: mismos nombres de clase, mismas firmas de metodo, mismos campos. No incluye clases que se llegaron a planificar pero no se implementaron (por ejemplo, una segunda fuente privada).

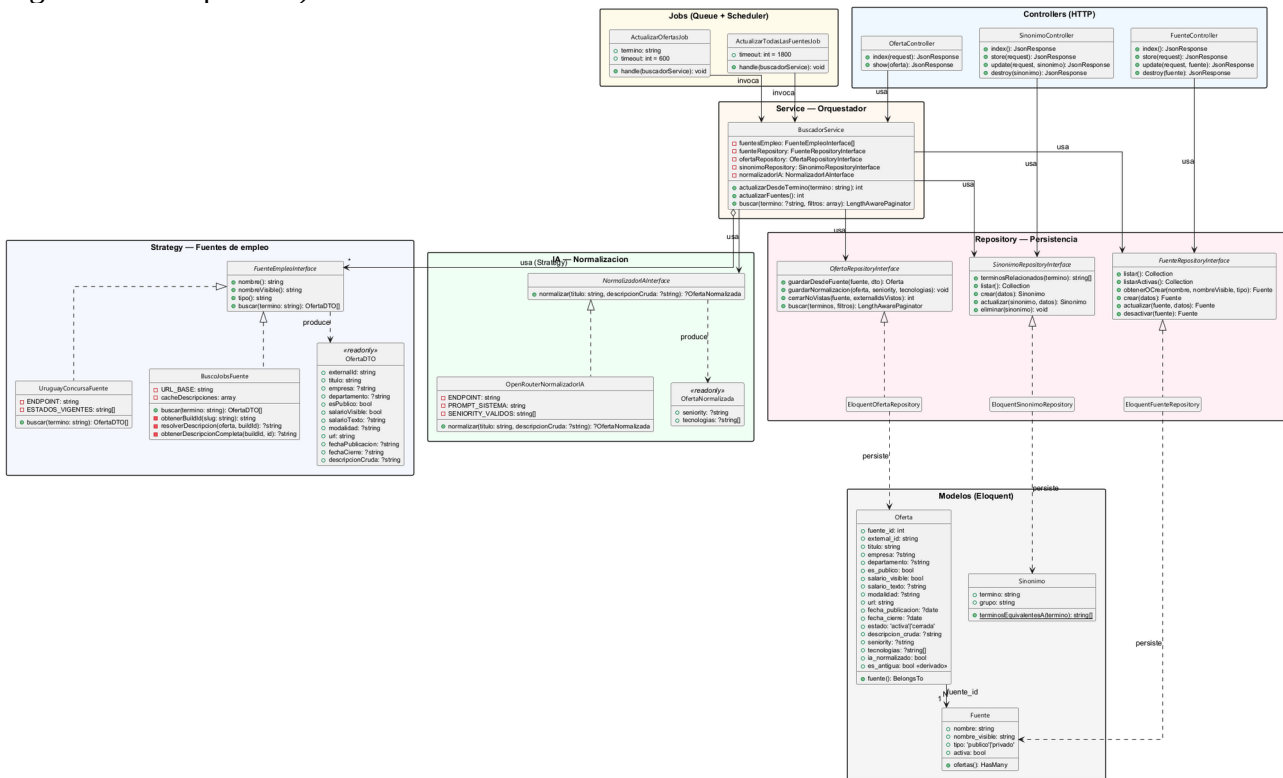


Diagrama de clases actualizado: Strategy (fuentes), Service, IA, Repository, Modelos, Controllers y Jobs, con los nombres y metodos reales del código.

Lectura del diagrama: `FuenteEmpleadoInterface` y sus dos implementaciones (`UruguayConcursoFuente`, `BuscoJobsFuente`) son el punto de extensión del sistema — sumar un portal nuevo es crear una clase mas, sin tocar `BuscadorService`. `NormalizadorIAInterface` (con `OpenRouterNormalizadorIA` como unica implementacion) es el unico punto de contacto con el modelo de lenguaje. Los tres repositorios (`Oferta`, `Fuente`, `Sinonimo`) son el ABM real del sistema: reemplazan el CRUD de usuarios que esta opcion de la consigna explicitamente no pide, administrando en su lugar el propio motor de busqueda (fuentes activas y sinonimos). `ActualizarOfertasJob` y `ActualizarTodasLasFuentesJob` corren en segundo plano (Queue + Scheduler), evitando consultar las fuentes externas en vivo en cada busqueda del usuario y aislando la falla de una fuente puntual.

7. Flujo general del sistema

UTalent separa por completo dos caminos que en un scraper ingenuo suelen mezclarse: el camino de actualizar datos (lento, depende de sitios externos, corre en segundo plano) y el camino de buscar (rapido, siempre local, nunca depende de un tercero).

7.1 Camino de actualizacion (en segundo plano)

Scheduler o comando manual → Job en la cola → BuscadorService → una FuenteEmpleoInterface por fuente activa → NormalizadorIA (opcional) → Repository → base de datos. Este camino nunca lo dispara una visita de un usuario: corre solo, cada 6 horas, o a pedido de un comando de consola.

7.2 Camino de busqueda del usuario (siempre local)

Navegador → vista Blade (solo estructura) → JavaScript de esa pagina → Axios → API (/api/ofertas) → OfertaController → BuscadorService::buscar() → OfertaRepository → base de datos → JSON → JavaScript pinta el resultado. En ningun punto de este camino se consulta Uruguay Concurso ni BuscoJobs: eso ya paso antes, en el camino de actualizacion. Por eso una busqueda del usuario final responde en milisegundos y nunca puede fallar porque un sitio externo este caido en ese momento.

8. Flujo de actualizacion de ofertas

Existen dos operaciones distintas en BuscadorService, con consecuencias distintas:

8.1 actualizarDesdeTermino(termino) — un termino puntual

Recorre las fuentes activas, le pide a cada una buscar(\$termino), guarda cada resultado (crea o actualiza segun fuente_id + external_id) y dispara la normalizacion por IA de las ofertas nuevas. No cierra ninguna oferta. La dispara ActualizarOfertasJob, y el comando buscador:actualizar {termino} para uso manual.

8.2 actualizarFuentes() — barrido completo

Recorre TODOS los terminos semilla configurados (config/buscador.php) para cada fuente activa, acumulando que external_id se vio en cada una. Recien cuando termino con todos los terminos de una fuente, si ninguno fallo, cierra (estado = 'cerrada') las ofertas de esa fuente que no aparecieron en ningun termino. Es la unica operacion que cierra ofertas (ver seccion 17). La dispara el Scheduler cada 6 horas, y el comando buscador:actualizar-todo para uso manual.

8.3 Por que guardar es siempre updateOrCreate

Cada oferta se identifica por el par (fuente_id, external_id). Volver a ver la misma oferta en una corrida posterior actualiza la fila existente en vez de duplicarla, lo que hace todo el proceso idempotente: correr el mismo Job dos veces (por ejemplo, si se reintenta tras un timeout) nunca genera datos duplicados.

9. Queue y Scheduler

QUEUE_CONNECTION=database: los Jobs se encolan en la tabla `jobs` y los procesa un worker (php artisan queue:work o queue:listen) que tiene que estar corriendo. Sin un worker activo, un Job despachado queda en cola sin ejecutarse — esto hay que tenerlo presente para cualquier demo en vivo.

- ActualizarOfertasJob: timeout 600s, 3 intentos, backoff 30/120/300s. Pensado para un termino puntual.
- ActualizarTodasLasFuentesJob: timeout 1800s, mismos reintentos. Es mas largo porque recorre todos los terminos semilla de todas las fuentes, incluyendo el enriquecimiento opcional de descripcion de BuscoJobs (ver seccion 18): un barrido real y completo, medido en la practica, tardo 79.73 segundos — muy por debajo del limite de 1800.
- DB_QUEUE_RETRY_AFTER=3600: tiene que superar con margen el timeout mas largo (1800s) para que la cola no de por perdido un Job que sigue corriendo y lo tome un segundo worker en paralelo.
- Scheduler (routes/console.php): Schedule::job(new ActualizarTodasLasFuentesJob())->everySixHours(). Un unico Job programado, no uno por termino, porque el cierre de ofertas necesita ver el resultado de todos los terminos antes de decidir que una oferta ya no existe.
- Dos comandos de consola para uso manual, sin esperar al Scheduler: `php artisan buscador:actualizar {termino}` y `php artisan buscador:actualizar-todo` (ambos solo despachan el Job; sigue haciendo falta `queue:work` para que se ejecute).

10. API

Toda la API vive bajo `/api/*` y responde siempre JSON, incluidos los errores de validacion: el middleware `ForzarRespuestaJson` fija la cabecera `Accept` antes de que la peticion llegue al controlador, para que un cliente que no la mande igual reciba un 422 con el detalle del error en vez de un redirect HTML pensado para navegador. Ademas, `/api/*` tiene un limite nativo de 60 peticiones por minuto por IP (`RateLimiter` de Laravel, sin dependencias nuevas).

10.1 Busqueda (solo lectura de datos locales)

- `GET /api/ofertas` — busqueda con filtros y paginacion. Nunca consulta una fuente externa ni despacha un Job.
- `GET /api/ofertas/{oferta}` — detalle de una oferta puntual. A diferencia del listado, devuelve la oferta este activa o cerrada (para poder mostrar “ya no esta disponible” en vez de un 404 para algo que existio). Una oferta que nunca existio si devuelve 404.

10.2 ABM (fuentes y sinonimos)

- `GET/POST/PUT/DELETE /api/fuentes` — alta, baja logica (desactivar, no borrar) y edicion de fuentes de empleo. El nombre tecnico de una fuente es inmutable una vez creada.
- `GET/POST/PUT/DELETE /api/sinonimos` — alta, baja y edicion de sinonimos de busqueda.

11. Buscador y filtros

El endpoint de búsqueda combina un termino de titulo (expandido por sinonimos) con hasta cuatro filtros independientes, todos opcionales y combinables con AND: sector (es_publico), salario visible (salario_visible), modalidad y departamento. Solo se aplica un filtro si el cliente realmente lo mando: un booleano que llega en “false” se distingue de un booleano que no llego, usando Request::has() en vez de Request::filled() para ese caso.

11.1 Por que no existe un filtro literal de “postulacion online”

La consigna menciona, como ejemplo de filtro posible, “si se puede postular online”. UTalent no implemento ese filtro tal cual, de forma deliberada: postulacion online y modalidad laboral son dos conceptos distintos. Postulacion online describe el metodo para aplicar a una oferta (llenar un formulario en la web del portal); modalidad laboral describe donde se trabaja una vez conseguido el puesto (remoto, hibrido o presencial). Uruguay Concurso no expone en ningun campo del dato consumido la modalidad laboral real de un llamado — todo lo que se puede afirmar de un llamado publico es que se posula por el portal, no si el puesto resultante es remoto, hibrido o presencial. Por eso, para las ofertas de Uruguay Concurso, el campo modalidad se guarda como null en vez de inventar un valor como “Remoto” que la fuente nunca informo. El filtro de modalidad existe (Remoto/Hibrido/Presencial) y funciona correctamente para BuscoJobs, que si expone esa informacion; para Uruguay Concurso, una oferta con modalidad null simplemente no coincide con ningun valor del filtro, que es el comportamiento honesto dado el dato disponible. Este comportamiento esta cubierto por tests automatizados (UruguayConcursoFuenteTest, OfertaBusquedaApiTest).

12. Sinonimos

La tabla sinonimos agrupa terminos equivalentes bajo un mismo “grupo” (por ejemplo, “programador” y “developer” en el grupo “programador”). Al buscar un termino, `Sinonimo::terminosEquivalentesA()` devuelve todos los terminos del mismo grupo (o solo el termino original si no esta registrado), y esa lista es la que se usa para filtrar por titulo. El termino se normaliza a minusculas antes de guardarse, para que la comparacion nunca dependa de como se escribio originalmente. Los sinonimos se administran desde el panel de administracion (`/admin/sinonimos`), no desde la busqueda publica.

13. NormalizadorIA

NormalizadorIAInterface define un unico metodo, `normalizar(titulo, descripcionCruda)`, que devuelve seniority (junior/semi_senior/senior o null) y una lista de tecnologias detectadas, o null si no se pudo normalizar. Su unica implementacion real, `OpenRouterNormalizadorIA`, es la unica clase de todo el proyecto que conoce el endpoint, la API key, el modelo y el formato de peticion/respuesta de OpenRouter (`openrouter.ai`). Se detalla en profundidad, junto con el uso de IA como herramienta de desarrollo, en la seccion 20.

Contrato de resiliencia: esta clase nunca lanza una excepcion hacia quien la llama ante un fallo esperable del proveedor (sin API key configurada, timeout, error HTTP, respuesta que no es un JSON valido). En cualquiera de esos casos registra el problema en el log y devuelve null;

BuscadorService guarda la oferta igual, sin enriquecer, y la deja con `ia_normalizado=false` para reintentarla en una proxima actualizacion.

14. Frontend

El frontend se construyo con Blade + Tailwind CSS 4 + JavaScript en modulos ES + Axios + Vite, sin ningun framework de componentes (React, Vue, Alpine, Livewire, Inertia). La regla de diseño que se siguio en las cinco vistas es la misma: Blade entrega unicamente la estructura HTML de la pagina; todo dato de ofertas, fuentes o sinonimos lo carga el JavaScript de esa pagina consultando la API (/api/*) con Axios. Ningun controlador de Laravel inyecta ese tipo de dato directamente hacia una vista.

14.1 Paginas

- / — home, con buscador y presentacion del producto.
- /ofertas — listado con filtros, paginacion y estados de carga/vacio/error.
- /ofertas/{id} — detalle de una oferta.
- /admin/fuentes y /admin/sinonimos — panel de administracion, sin autenticacion, accesible solo por URL directa (no enlazado desde la navegacion publica).

14.2 Organizacion del JavaScript

Vite arma un punto de entrada por pagina (bundles pequeños y enfocados en vez de un unico bundle gigante). api.js es el unico archivo de todo el frontend que sabe construir una URL de /api/*: ninguna otra pagina arma una peticion a mano. utilidades.js concentra el formateo compartido entre paginas (badges de sector, modalidad y oferta antigua, texto de salario/empresa, escape de HTML antes de insertar cualquier dato externo en el DOM).

14.3 Estado por URL

Los filtros de busqueda se reflejan en la URL via URLSearchParams + history.pushState (sin router de cliente): una busqueda se puede compartir o recargar sin perder los filtros aplicados.

15. Accesibilidad y diseño responsive

Tras completar el frontend funcional se hizo una auditoria especifica de UX/UI/accesibilidad/rendimiento (analisis primero, sin tocar codigo; implementacion despues, con cada cambio verificado en navegador real). De esa auditoria surgieron y se corrigieron:

- Contraste insuficiente: los 3 titulos de tarjeta del home usaban un turquesa claro sobre blanco (1.86:1, por debajo del minimo de WCAG AA). Se agrego una variante mas oscura del mismo color (--color-accent-text, 5.47:1) exclusiva para texto sobre fondo claro, sin sumar un color ajeno a la paleta.
- Foco de teclado inconsistente: cada navegador mostraba su propio foco por defecto, sin relacion con la paleta del sitio. Se unifico con un anillo visible propio (outline de 2px en el mismo color del punto anterior) para links, botones, inputs, selects y el acordeon de filtros mobile.
- El boton de menu mobile actualizaba visualmente el menu pero nunca sincronizaba aria-expanded: quedaba en "false" aunque el menu estuviera abierto. Se corrigio para que el atributo represente siempre el estado real.
- La pagina de resultados desplazaba el scroll hacia la lista incluso en la primera carga (antes de que el usuario hiciera nada). Se limito el desplazamiento automatico a acciones reales del usuario (buscar, filtrar, paginar), nunca a la carga inicial.
- Verificado manualmente en 375px, 768px, 1024px y 1440px de ancho: sin scroll horizontal, sin elementos cortados, con el acordeon de filtros (<details>/<summary> nativo, sin JavaScript) funcionando en mobile.

16. Manejo de errores y resiliencia

El principio general es que la falla de una parte del sistema nunca debe tumbar a las demas. Se aplica en tres niveles:

- Entre fuentes: si UruguayConcursaFuente o BuscoJobsFuente lanzan una excepcion al buscar un termino, BuscadorService la captura, la registra en el log y sigue con la fuente siguiente. La otra fuente no se entera de la falla.
- Entre la fuente y la IA: si NormalizadorIA falla (sin API key configurada, timeout, error HTTP, JSON invalido), nunca lanza una excepcion: devuelve null, y la oferta se guarda de todas formas sin enriquecer.
- Dentro de una misma fuente, por oferta puntual: en BuscoJobsFuente, si la peticion opcional de detalle (descripcion completa) falla para una oferta (404, 500, timeout, JSON invalido, o el campo esperado ausente), se registra un warning especifico y esa oferta se guarda igual con la descripcion truncada que ya se tenia. Las demas ofertas del mismo termino no se ven afectadas.
- Cierre de ofertas: si un termino falla durante un barrido completo, esa fuente no cierra ninguna oferta en esa corrida (no se puede afirmar “ya no existe” con datos parciales), pero lo que si se pudo traer de otros terminos se guarda igual.

Cada uno de estos comportamientos tiene al menos un test automatizado dedicado (ver seccion 19): la resiliencia no es una afirmacion sin verificar, es un comportamiento reproducible.

17. Cierre de ofertas y por que requiere un barrido completo

El primer diseño de este mecanismo se descarto antes de programarlo por un motivo concreto: cerrar una oferta simplemente porque no aparecio en la busqueda de UN termino es incorrecto. Una oferta de “Analista de Datos” puede no aparecer al buscar “Programador” sin que eso signifique que la oferta ya no existe: solo significa que ese termino puntual no la encontro.

La solucion implementada (`OfertaRepositoryInterface::cerrarNoVistas`) exige ver el resultado de TODOS los terminos semilla de una fuente antes de decidir que una oferta especifica ya no existe. `BuscadorService::actualizarFuentes()` acumula, por fuente, el conjunto completo de `external_id` vistos en cualquiera de los terminos; recien al terminar de recorrerlos todos, si ninguno fallo, cierra las ofertas activas de esa fuente cuyo `external_id` no quedo en ese conjunto. Dos salvaguardas adicionales: si algun termino fallo durante el barrido, no se cierra nada de esa fuente en esa corrida (evita cerrar por datos incompletos); y si el conjunto de vistos llegara vacio por algun error de programacion, el repositorio se niega a cerrar nada (evita borrar de un plumazo el historial completo de una fuente).

Importante: cerrar una oferta nunca borra el registro, solo cambia su estado a 'cerrada'. El historico se conserva y sigue siendo consultable en el detalle (que muestra “ya no esta disponible” en vez de ocultarla). Este mecanismo esta cubierto en detalle por `BuscadorServiceCierreOfertasTest`.

17.1 Antigüedad de una oferta: señal visual, no cierre

Aparte, y sin relacion con el cierre: algunas ofertas publicas siguen siendo reportadas como vigentes por la fuente original durante años (se verificaron en produccion llamados de hasta 2018 aun activos en Uruguay Concurso). Cerrarlas automaticamente por antigüedad seria inventar un criterio que la fuente no avala. En cambio, `Oferta::es_antigua` es un atributo derivado (no una columna) que compara la fecha de publicacion contra un umbral de 180 dias y solo se usa para mostrar una etiqueta “Oferta antigua” en el frontend: nunca cambia estado ni afecta si la oferta aparece o no en una busqueda.

18. Segunda fuente: demostracion real de Strategy

Con una sola fuente, el patron Strategy no se pone a prueba: cualquier diseño “funciona” si solo hay una implementacion. La segunda fuente (BuscoJobsFuente) se eligio y se implemento especificamente para verificar que agregar un sitio con un mecanismo de datos completamente distinto no requiriera tocar BuscadorService, OfertaRepository ni ningun otro colaborador. Diferencias reales entre ambas fuentes, ambas resueltas exclusivamente dentro de la clase correspondiente:

- Uruguay Concurso: una unica peticion POST a un endpoint JSON fijo, con paginado y filtro de años propios del sitio.
- BuscoJobs: dos peticiones encadenadas (pagina de busqueda para extraer un buildId que cambia con cada despliegue del sitio, y luego el endpoint de datos con ese id), mas una tercera peticion opcional por oferta para completar la descripcion cuando el listado la trae truncada (con cache en memoria por IdOferta para no repetir esa tercera peticion dos veces en el mismo barrido si la misma oferta aparece bajo mas de un termino semilla).
- Uruguay Concurso no informa modalidad laboral real (ver seccion 11.1); BuscoJobs si, con una regla de prioridad propia cuando el sitio marca una oferta como apta para teletrabajo e hibrido a la vez (gana “Remoto”, por ser la categoria mas amplia).

La prueba mas directa de que el patron cumple su proposito es

BuscadorServiceMultiplesFuentesTest: instancia un unico BuscadorService con ambas Strategy reales inyectadas, sin ningun condicional en el Service que distinga una de otra, y verifica que las ofertas de ambas se guarden correctamente y sin colisionar entre si.

19. Tests y validaciones

82 tests automatizados (PHPUnit) en 19 archivos, ejecutados con `php artisan test`, corriendo en la base de datos SQLite en memoria configurada para el entorno de testing (no toca database.sqlite). Se agrupan así:

- Fuentes externas (Strategy): BuscoJobsFuenteTest (mapeo de campos, modalidad, resiliencia del detalle opcional) y UruguayConcursaFuenteTest (modalidad null).
- Orquestación y resiliencia: BuscadorServiceMultiplesFuentesTest, BuscadorServiceResilienciaTest, BuscadorServiceCierreOfertasTest (el más importante para el mecanismo de la sección 17), BuscadorServiceNormalizacionTest.
- IA: OpenRouterNormalizadorIATest.
- API de búsqueda y detalle: OfertaBusquedaApiTest (filtros, sinónimos, paginación, ofertas antiguas), OfertaDetalleApiTest.
- ABM y Repository: FuenteApiTest, SinonimoApiTest, FuenteDesactivacionFlujoTest, SinonimoTest.
- Jobs y comandos: ActualizarOfertasCommandTest.
- Infraestructura transversal: RateLimitApiTest, OfertaActualizacionDatosTest (idempotencia del guardado).
- OfertaAntiguedadTest: umbral de los 180 días y que ser antigua nunca cambia el estado de la oferta.

19.1 Que no tiene test automatizado, y por que

No hay ningún test automatizado de frontend (JavaScript): el proyecto no incorporo ningún framework ni runner de testing para esa capa (decisión de alcance, coherente con no sumar dependencias que el proyecto no necesita). Esa capa se valida siempre de forma manual, en navegador real, en cada fase que la toco (incluyendo las cuatro resoluciones de la sección 15). El contraste de color tampoco se puede verificar con PHPUnit: se midió con cálculo manual del contraste WCAG (fórmula de luminancia relativa) y se confirmó visualmente en navegador.

20. Uso de Inteligencia Artificial (Ejercicio 3)

El ejercicio 3 de la consigna pide documentar el uso de IA en el trabajo, aclarando modelo y proposito, y admite explicitamente que usarla no resta puntos siempre que se pueda identificar y modificar ese codigo en clase. Esta seccion separa los dos usos de IA que tuvo este proyecto, que son de naturaleza distinta: una es una funcionalidad del propio producto (A), la otra fue una herramienta usada durante el desarrollo (B).

A) IA como parte del producto: NormalizadorIA + OpenRouter

Proveedor: OpenRouter (openrouter.ai). Configuración del proyecto: el modelo se parametriza por completo via `OPENROUTER_MODEL` en `.env`; el código no fija ningún modelo específico, cualquiera compatible con el formato de chat completions de OpenRouter sirve. Modelo utilizado durante las pruebas: ninguno con credenciales reales — el desarrollo de esta clase se verificó siempre con `Http::fake()` (ver `OpenRouterNormalizadorIA`Test, que simula la respuesta del proveedor sin llamarlo), y el único barrido real corrido contra las fuentes (Fase 9.2, sección 9) se hizo deliberadamente sin `OPENROUTER_API_KEY` configurada, precisamente para confirmar en la práctica que el sistema sigue funcionando sin ese enriquecimiento. En consecuencia, nunca se ejecuto este componente contra el proveedor real con un modelo concreto durante este proyecto. Que datos normaliza: a partir del título y la descripción cruda de una oferta ya guardada, se le pide al modelo que devuelva únicamente un objeto JSON con dos campos: `seniority` (junior/semi_senior/senior, o null si no se puede determinar) y `tecnologias` (una lista de strings, o vacía). La respuesta se valida estrictamente antes de guardarse: un valor de `seniority` que no sea exactamente uno de los tres esperados se descarta como null, y la lista de `tecnologias` se filtra, se recorta a 15 elementos y se le quitan duplicados.

Por que se usa: las fuentes originales publican esta información, si existe, mezclada en texto libre dentro de la descripción. Extraerla de forma estructurada permite en el futuro filtrar u ordenar por `seniority` o tecnología sin depender de que el usuario lea la descripción completa de cada oferta. Que pasa cuando falla: `OpenRouterNormalizadorIA` es la única clase de todo el proyecto que conoce este proveedor (ver sección 13). Ante cualquier fallo esperable (sin API key configurada, timeout, error HTTP, respuesta que no es JSON válido) nunca lanza una excepción: registra el problema en el log y devuelve null. La oferta se guarda igual, sin `seniority` ni tecnologías, y queda marcada para reintentarse en la próxima actualización. Verificado en la práctica: el barrido real de la Fase 9.2 (79.73 segundos, sección 9) se corrió sin ninguna `OPENROUTER_API_KEY` configurada, y las ofertas se guardaron con normalidad, solo sin ese enriquecimiento.

B) IA como herramienta de desarrollo: Claude Code

Además de la IA integrada al producto, el desarrollo de UTalent se hizo con la asistencia de Claude Code (Anthropic), un asistente de línea de comandos basado en un modelo de lenguaje grande (Claude), usado como herramienta de pair-programming durante todas las fases del proyecto. Se documenta acá de forma honesta y sin exagerar su aporte, tal como pide el ejercicio 3.

Como se utilizó, en la práctica:

- El proyecto se dividió en fases (scaffolding inicial, primera fuente, Queue/Job/Scheduler, ABM, endpoint de búsqueda, IA de normalización, auditoría y endurecimiento del backend, segunda fuente, frontend, auditoría de UX/accesibilidad, correcciones, mejora de la descripción de BuscoJobs, y esta misma preparación de entrega). Antes de programar cada fase, el asistente proponía un plan concreto (que clases, que decisiones, que se iba a verificar y como); el estudiante lo aprobaba, lo ajustaba o lo rechazaba antes de que se escribiera una sola línea de código.
- Cada fase se cerró con verificación real, no solo con la palabra del asistente: llamadas HTTP reales contra Uruguay Concurso y BuscoJobs (con curl y con el navegador, inspeccionando la

pestaña de red antes de programar cada Strategy), la suite de tests completa corriendo en verde, y pruebas manuales en navegador real en varias resoluciones para todo lo relacionado a frontend.

- El asistente tambien se uso para tareas de analisis puro, sin escribir codigo: por ejemplo, la auditoria de UX/accesibilidad (seccion 15) y esta misma auditoria de entrega academica se pidieron explicitamente como analisis, con la instruccion de no tocar codigo hasta tener una propuesta aprobada.

Como se revisaron y validaron las respuestas:

- Ninguna fase paso a la siguiente sin aprobacion explicita del estudiante. Varias propuestas se ajustaron antes de aprobarse: por ejemplo, el mecanismo de cierre de ofertas (seccion 17) se replanteo por advertencia directa del estudiante antes de programarse, señalando que cerrar por un termino aislado era incorrecto.
- Cuando una respuesta del modelo de IA (ya sea del asistente de desarrollo o, en pruebas, de NormalizadorIA) resultaba dudosa, se verifico contra el comportamiento real del sistema en vez de aceptarla por confianza: por ejemplo, el hallazgo de foco de teclado “ausente” de la primera auditoria de accesibilidad resulto ser un error de metodo de prueba (un foco programatico por JavaScript no dispara :focus-visible igual que una tecla Tab real); se detecto probando con teclado real en el navegador y se corrigio el diagnostico antes de decidir que arreglar.

Que se aprendio en el proceso:

- A dirigir un desarrollo asistido por IA con especificaciones precisas en vez de pedidos vagos: cuanto mas exacto el pedido (que clase, que restriccion de arquitectura, que se debia verificar antes de dar por cerrada una fase), mejor el resultado y menos iteraciones hicieron falta.
- A no confiar una afirmacion de la IA sin verificarla: los pasos de verificacion real (peticiones HTTP reales, tests automatizados, pruebas manuales en navegador) fueron una exigencia constante del estudiante en cada fase, no un paso opcional.
- Contenido tecnico propiamente dicho: el patron Strategy resuelve el problema de fuentes heterogeneas; por que separar Repository de la logica de negocio facilita que la busqueda nunca dependa de una fuente externa en vivo; por que un mecanismo de cierre necesita ver el conjunto completo antes de decidir; como reconocer, inspeccionando la pestaña de red de un navegador, el endpoint interno que un sitio usa para si mismo en vez de asumir que hace falta scraping de HTML.

Que decisiones tomo el estudiante (no la IA):

- La eleccion del proyecto (Opcion 3) y del stack (Laravel).
- La aprobacion o el rechazo de cada propuesta de arquitectura antes de programarse, incluyendo pedir explicitamente que se replantee el mecanismo de cierre de ofertas antes de aceptarlo.
- El alcance de cada fase (por ejemplo, limitar la Fase 9.2 a mejorar solo la descripcion de BuscoJobs y dejar expresamente afuera otros campos disponibles en el mismo endpoint, como requisitos o rango salarial).
- Cuando una fase quedaba aprobada y se podia pasar a la siguiente.

Aporte al proyecto de egreso: el proyecto de egreso del estudiante no es este mismo tema, por lo que el aporte no es de contenido sino de metodo: experiencia real dirigiendo un desarrollo con Laravel (framework que se usara probablemente en el proyecto de egreso o en el ambito laboral), con patrones de diseño aplicados a un problema real (no de manual) y con una disciplina de verificacion (tests automatizados, pruebas manuales, auditorias propias antes de entregar) que es independiente de que el desarrollo se haya apoyado en un asistente de IA o no.

21. Instalacion y ejecucion

Requisitos previos en la maquina:

- PHP 8.2 o superior, con la extension pdo_sqlite habilitada.
- Composer.
- Node.js y npm (para compilar el frontend con Vite).
- No hace falta instalar MySQL, PostgreSQL, Redis ni Memcached: la base de datos es un unico archivo SQLite y la cola usa el driver database.

21.1 Instalacion inicial

Desde la carpeta utalent/ del repositorio:

- composer setup

Ese unico comando (definido en composer.json) hace, en este orden: composer install, copia .env.example a .env si no existe, genera APP_KEY, corre las migraciones y compila el frontend (npm install + npm run build).

21.2 Generar datos reales antes de una demo

Un proyecto recién clonado no tiene ofertas: hace falta correr al menos una vez la actualización contra las fuentes reales. Como la cola usa el driver database (no sync), un Job despachado no se ejecuta solo: hace falta un worker corriendo.

- php artisan buscador:actualizar-todo (despacha el barrido completo)
- php artisan queue:work --once (procesa ese Job)

21.3 Levantar el proyecto para desarrollo o demo

Cualquiera de las dos opciones:

- composer dev — levanta en paralelo el servidor (php artisan serve), un worker de cola (queue:listen), el visor de logs (pail) y Vite en modo watch, todo en una sola terminal.
- O por separado: php artisan serve, y en otra terminal php artisan queue:work para que los Jobs se procesen.

21.4 Ejecutar la suite de tests

- php artisan test

22. Variables de entorno

El archivo `.env.example` (trackado en el repositorio) trae valores por defecto razonables para todo excepto la IA. Las que importan para entender el comportamiento del sistema:

- `DB_CONNECTION=sqlite` — base de datos en un unico archivo (`database/database.sqlite`).
 - `QUEUE_CONNECTION=database` — los Jobs se procesan por un worker, no en el mismo request (ver seccion 9).
 - `DB_QUEUE_RETRY_AFTER=3600` — debe superar el timeout mas largo entre los Jobs (1800s).
 - `OPENROUTER_API_KEY` y `OPENROUTER_MODEL` — opcionales. Sin configurar, el sistema sigue funcionando completo: guarda ofertas normalmente, solo sin enriquecer con seniority/tecnologias (verificado en la practica, no supuesto).
 - `APP_KEY` — se genera con `php artisan key:generate` (parte de composer setup), no se versiona.
- Ninguna de estas variables, ni el archivo `.env` real, se sube al repositorio (ver seccion 21 y el `.gitignore` del proyecto).

23. Limitaciones conocidas

- Solo dos fuentes de empleo (una publica, una privada). El diseño (Strategy) admite agregar mas sin tocar el resto del sistema, pero no se implemento una tercera por alcance y tiempo.
- Sin filtro de antigüedad: la señal “Oferta antigua” (seccion 17.1) es solo visual; no existe todavia una forma de excluir esas ofertas de una busqueda.
- La descripcion completa de BuscoJobs (seccion 18) depende de un endpoint interno no documentado oficialmente por ese sitio; si el sitio cambia su formato, esa mejora puntual podria dejar de funcionar sin afectar el resto del sistema (fallback automatico a la descripcion truncada, ver seccion 16).
- Ambas fuentes dependen de mecanismos internos no documentados oficialmente (identificados por inspeccion de red, no por una API publica): si cualquiera de los dos sitios cambia esos mecanismos, la Strategy correspondiente necesitaria ajustarse.
- Sin autenticacion en el panel de administracion (/admin/*): protegido solo por no estar enlazado desde la navegacion publica, no por un mecanismo de seguridad real. Aceptable para el alcance academico del practico (la consigna no pide autenticacion para esta opcion), pero no seria aceptable en un entorno expuesto a internet sin agregarla.
- Sin tests automatizados de frontend (JavaScript): esa capa se valida siempre de forma manual (ver seccion 19.1).
- Uruguay Concurso no expone modalidad laboral real: el filtro de modalidad no aplica a esa fuente (ver seccion 11.1), por diseño y no por un defecto pendiente de arreglar.

24. Conclusiones

UTalent cumple lo que pide la Opcion 3 de la consigna —busqueda de oportunidades laborales con filtros, agregando mas de una fuente, sin registro de usuarios ni postulacion desde el sistema— y va mas alla del MVP minimo: dos fuentes reales con mecanismos de datos distintos (demostrando el patron Strategy en la practica, no solo en el papel), actualizacion periodica en segundo plano con cierre correcto de ofertas que ya no aparecen en la fuente original, enriquecimiento opcional con IA, un panel de administracion real para el motor de busqueda, y un frontend propio revisado por accesibilidad y responsive.

El desarrollo se organizo en fases pequeñas, cada una con su propia verificacion real (llamadas HTTP reales contra las fuentes, tests automatizados, pruebas manuales en navegador) antes de darse por cerrada, incluyendo tres rondas dedicadas exclusivamente a auditar el propio trabajo (backend en la Fase 6, UX/accesibilidad en la Fase 9, y esta preparacion de entrega en la Fase 10/11) en vez de asumir que estaba bien por haber pasado los tests.

Quedan limitaciones conocidas y documentadas (seccion 23), ninguna de las cuales compromete lo que la consigna pide para esta opcion; son, en su mayoria, decisiones deliberadas de alcance o consecuencias esperables de depender de sitios externos sin API publica documentada.